Dynamic Programming Patterns

Based on Alvin Zablan · freeCodeCamp · Learn to Solve Algorithmic Problems

This guide covers six canonical dynamic programming problems arranged in two arcs: numeric DP (canSum → howSum → bestSum) and string DP (canConstruct → countConstruct → allConstruct). Each arc tightens the return constraint from a boolean decision through path reconstruction to exhaustive enumeration. Both memoization (top-down) and tabulation (bottom-up) strategies are developed for every problem, with complexity analysis and detailed gotchas.

Problem	Domain	Returns	Early Return?
`canSum`	`Numeric`	`bool`	Yes
`howSum`	`Numeric`	`list | None`	Yes
`bestSum`	`Numeric`	`shortest list`	No
`canConstruct`	`String`	`bool`	Yes
`countConstruct`	`String`	`int`	No
`allConstruct`	`String`	`list[list[str]]`	No

Arc 1 — Numeric DP

These three problems all operate on a single integer target sum. The subproblem is always the remaining sum after subtracting one number. The progression canSum → howSum → bestSum tightens the return constraint: from "can we?" to "how?" to "what is the shortest way?".

Problem 1 — canSum (Decision Problem)

Problem Statement: Given a `targetSum` (integer) and an array `numbers`, return `True` if it is possible to generate exactly `targetSum` by summing any combination of values from the array. Values may be reused any number of times.

Return type: Boolean

Intuition

Alvin models this as a **tree of remainders**. The root holds `targetSum`. Each directed edge subtracts one element from the array, producing a child node whose value is the new remainder. We ask: *does any path from root to leaf reach exactly 0?*

The two base cases are:

- `remainder = 0` → return `True` (success — we balanced out perfectly).
- `remainder < 0` → return `False` (overshot — this branch is dead).

Because we only need *one* valid path, we can **return early** the moment any child reports `True`. This is the key property distinguishing a decision problem from an optimisation problem.

Recursive Tree

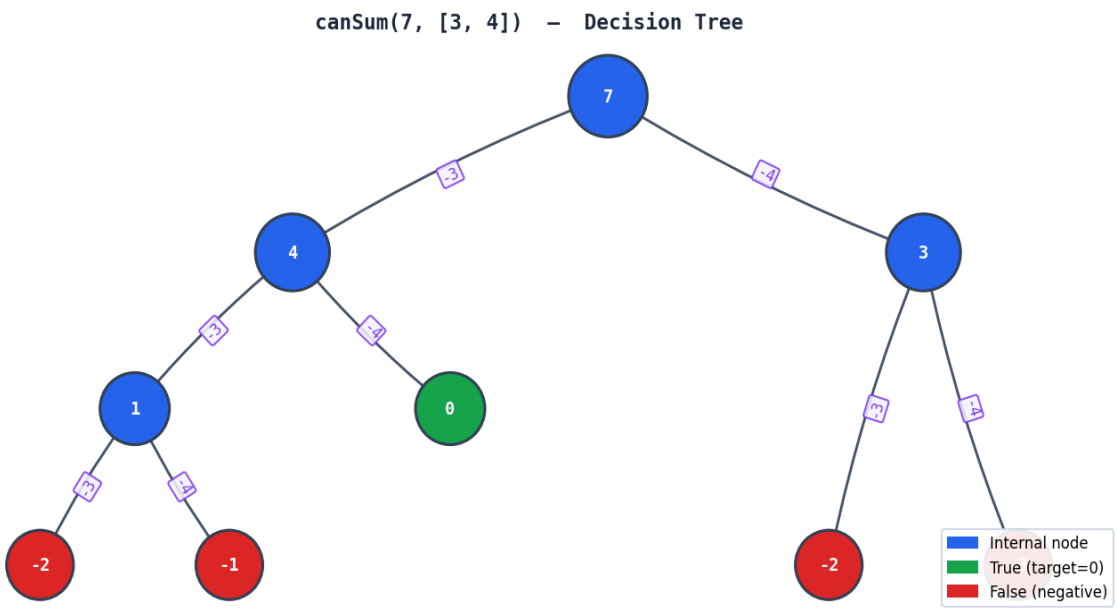


Figure 1. The decision tree for canSum(7, [3, 4]). Edge labels show the number subtracted. The green node (0) is the only success state; red nodes are dead ends.

Complexity Analysis

Without memoisation the tree has branching factor *n* (one branch per number) and maximum depth *m* (the target sum divided by the smallest number). This yields an exponential brute-force cost:

```
Brute-force: Time O(n^m) Space O(m)

Memoization: Time O(n · m) Space O(m)

Tabulation: Time O(n · m) Space O(m)


n = len(numbers), m = targetSum
```

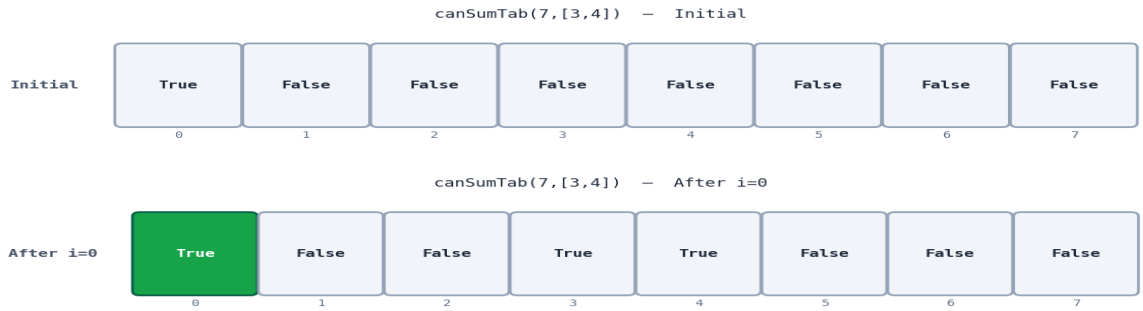
Memoisation reduces the unique subproblems from $O(n^m)$ to O(m) — one per distinct remainder value. Each subproblem then tries each of the *n* numbers once, giving the O(n·m) bound.

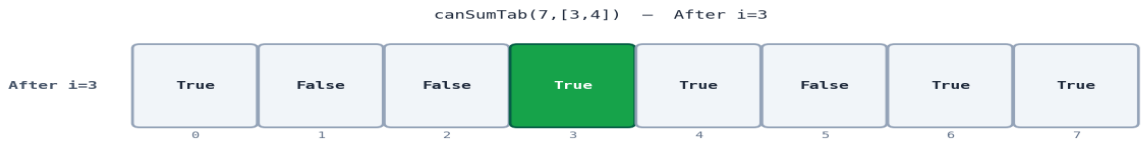
Memoization Implementation

```
def can_sum(target_sum, numbers, memo=None):

if memo is None: memo = {}

if target_sum in memo: return memo[target_sum] # cache hit

if target_sum == 0: return True # base case: success

if target_sum < 0: return False # base case: overshot


for num in numbers:

if can_sum(target_sum - num, numbers, memo):

memo[target_sum] = True

return True # early return: first True path wins


memo[target_sum] = False # all paths failed, cache the result

return False
```

Tabulation Walk-Through

We build a boolean array of size `targetSum + 1`, seeded with `True` at index 0. Iterating left-to-right, every `True` position "reaches forward" by each number, marking future indices reachable.





The highlighted cell is the current propagation position. After i=3 (True), both 3+3=6 and 3+4=7 get marked True.

```
def can_sum_tab(target_sum, numbers):
table = [False] * (target_sum + 1)
table[0] = True # seed: 0 is always reachable

for i in range(target_sum + 1):
if table[i]: # propagate from reachable positions
for num in numbers:
if i + num <= target_sum: # bounds guard
table[i + num] = True

return table[target_sum]
```

■ Common Gotchas

- Early return is valid ONLY because we need any True path. Remove it for bestSum.
- Bounds check (i + num <= target_sum) is mandatory — indices beyond the table cause IndexError.
- The memo key is the remainder integer. Never share a memo dict across independent top-level calls.
- Negative numbers in the array would cause infinite recursion. This problem assumes all values > 0.

Problem 2 — howSum (Combinatoric Problem)

Problem Statement: Return *any one* combination of numbers from the array that sums to `targetSum`, as a list. Return `None` if no combination exists.

Return type: `list | None`

What Changed from canSum?

The tree structure is identical. But instead of returning a Boolean, we must **reconstruct the path** taken to reach 0. When the base case `target=0` fires, it returns an empty list `[ ]`. Each ancestor on the call stack appends the number it chose (the edge label) to the list being bubbled up. By the time the result reaches the root, the list contains the full combination.

The critical Python trap: an empty list `[ ]` is **falsy**. Always check `if result is not None`, never `if result`.

Recursive Tree

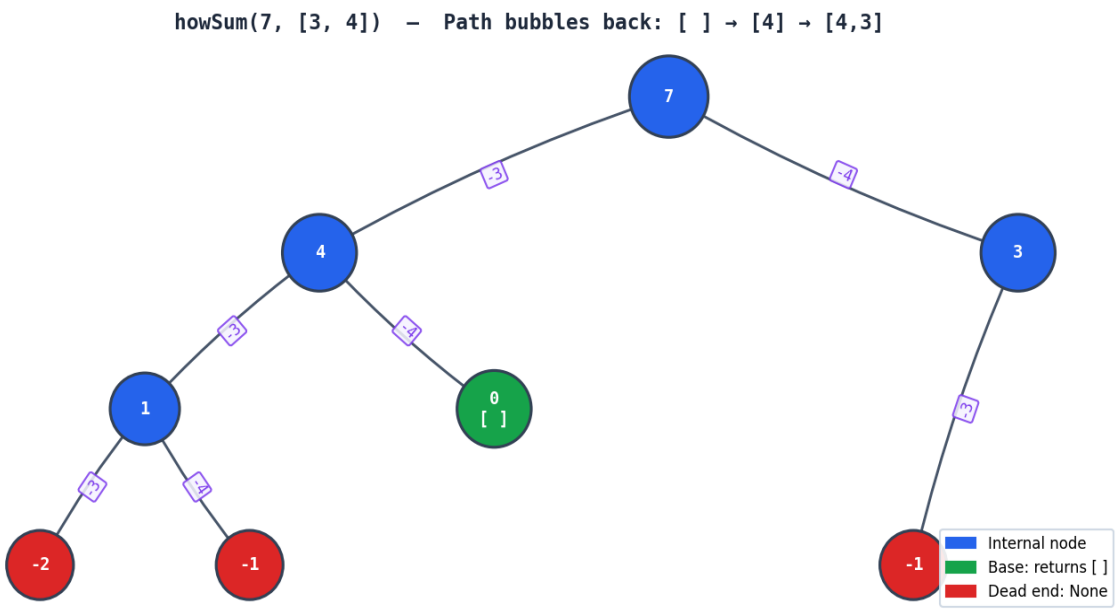


Figure 2. howSum(7, [3, 4]). The green leaf returns [ ]; its parent appends 4 to yield [4]; the grandparent appends 3 to yield [4, 3]. The combination is assembled on the way back up.

Complexity Analysis

Each recursive call may copy a list of up to *m* elements, adding an O(m) factor to every node. With O(n·m) nodes after memoisation:

```
Brute-force: Time O(n^m · m) Space O(m²)

Memoization: Time O(n · m²) Space O(m²)

Tabulation: Time O(n · m²) Space O(m²)
```

Memoization Implementation

```
def how_sum(target_sum, numbers, memo=None):
```

```
if memo is None: memo = {}
if target_sum in memo: return memo[target_sum]
if target_sum == 0: return [] # success: empty combination
if target_sum < 0: return None # overshot

for num in numbers:
result = how_sum(target_sum - num, numbers, memo)
if result is not None: # found a valid path
combination = [*result, num] # append this edge's number
memo[target_sum] = combination
return combination # early return: first path wins

memo[target_sum] = None
return None
```

Tabulation Walk-Through

The table stores `None` (unreachable) or a list (the path). At each reachable position *i*, we copy the current path and append `num` before writing it to `table[i+num]`.

```
def how_sum_tab(target_sum, numbers):
table = [None] * (target_sum + 1)
table[0] = [] # seed: empty combo reaches 0

for i in range(target_sum + 1):
if table[i] is not None:
for num in numbers:
if i + num <= target_sum:
table[i + num] = [*table[i], num] # copy + append

return table[target_sum]
```

■ Common Gotchas

- if result is not None — NOT if result. An empty list [] is a valid success value but is falsy.
- Always copy the list: [*result, num]. Mutating in place corrupts memo-cached references.
- The number is appended after the child's list — the combination reads reverse-chronologically.

Problem 3 — bestSum (Optimisation Problem)

Problem Statement: Return the **shortest** combination of numbers that adds to `targetSum`. Return `None` if impossible.

Return type: `list | None` (shortest length wins)

The Critical Difference — No Early Return

In `howSum` we returned immediately on finding the *first* valid path. Here we must visit **every** leaf, because the first path found is not necessarily the shortest. Alvin introduces a `shortest` variable initialised to `None`. After each successful branch, we compare lengths and keep only the shorter result.

Recursive Tree

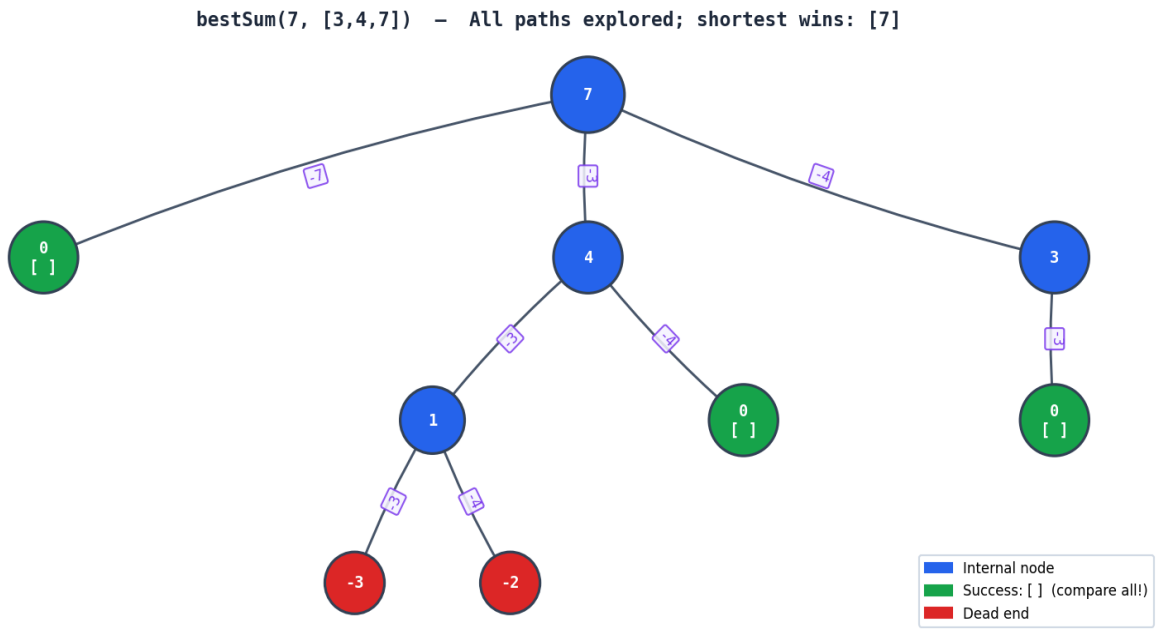


Figure 3. bestSum(7, [3,4,7]). Three paths reach a leaf: [7] (length 1), [3,4] (length 2), [4,3] (length 2). All must be explored before [7] is confirmed shortest.

Complexity Analysis

Exhaustive search does not change the asymptotic bound — the dominant $O(n \cdot m^2)$ factor comes from list copying, identical to howSum.

```
Brute-force: Time O(n^m · m) Space O(m²)

Memoization: Time O(n · m²) Space O(m²)

Tabulation: Time O(n · m²) Space O(m²)
```

Memoization Implementation

```
def best_sum(target_sum, numbers, memo=None):

if memo is None: memo = {}

if target_sum in memo: return memo[target_sum]
```

```
if target_sum == 0: return []
if target_sum < 0: return None

shortest = None # sentinel: no winner yet

for num in numbers:
result = best_sum(target_sum - num, numbers, memo)
if result is not None:
combination = [*result, num]
# Update only if this is the first OR strictly shorter
if shortest is None or len(combination) < len(shortest):
shortest = combination # NO early return!

memo[target_sum] = shortest # cache the best for this remainder
return shortest
```

Tabulation — Conditional Update

In tabulation, we write to `table[i+num]` only if the candidate combination is strictly shorter than whatever is already stored there:

```
def best_sum_tab(target_sum, numbers):
table = [None] * (target_sum + 1)
table[0] = []

for i in range(target_sum + 1):
if table[i] is not None:
for num in numbers:
if i + num <= target_sum:
candidate = [*table[i], num]
future = table[i + num]
# Replace only if future is empty or candidate is shorter
if future is None or len(candidate) < len(future):
table[i + num] = candidate

return table[target_sum]
```

■ Common Gotchas

- No early return — removing it is the entire point of bestSum over howSum.
- shortest = None as sentinel. Initialising to [] would collide with the real base case.
- Use strict < not <=. Using <= overwrites with equal-length paths unnecessarily.
- Pattern: canSum (bool) → howSum (any path) → bestSum (optimal path). Recognise this arc on new problems.

Arc 2 — String DP

The same three-level progression applied to strings. The subproblem is a suffix of the original target string. The prefix rule replaces the negativity check: a word is only valid if it matches the very beginning of the current string.

Problem 4 — canConstruct (String Decision Problem)

Problem Statement: Given a string `target` and a list of strings `wordBank`, return `True` if `target` can be constructed by concatenating words from the bank. Words may be reused.

Return type: Boolean

The Numeric → String Parallel

Numeric DP	String DP
`targetSum (int)`	`target (string)`
`Subtract a number`	`Remove a prefix word`
`Remainder (int)`	`Suffix (string)`
`Base: remainder = 0`	`Base: target = ""`
`Base: remainder < 0`	`Base: no prefix match`

The Prefix Rule

Alvin's most important warning for this problem: **a word can only be used if it matches the very start (prefix) of the current target**. It is tempting to search for the word anywhere in the string, but pulling a word from the middle leaves disconnected fragments that cannot be legally concatenated. Check: `target.startswith(word)`, then slice off that prefix to obtain the suffix for the recursive call.

Recursive Tree

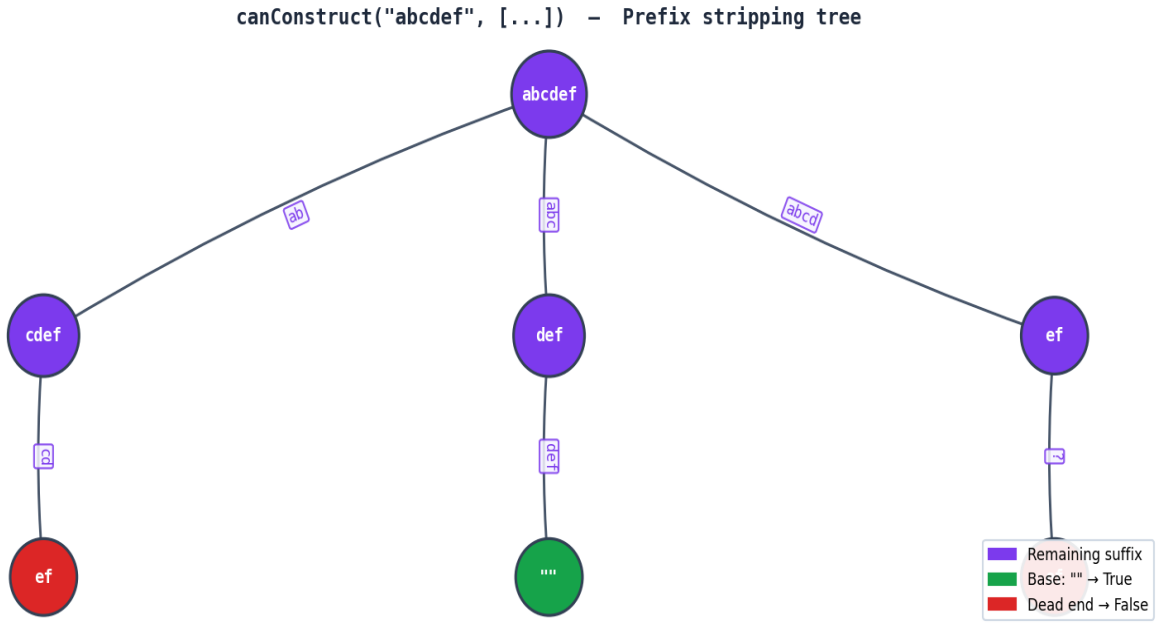


Figure 4. canConstruct("abcdef", ["ab","abc","cd","def","abcd"]). Edge labels show the prefix removed. The path "abc"→"def"→"" succeeds.

Complexity Analysis

Let m = `len(target)`, n = `len(wordBank)`. String slicing costs O(m) per call, adding one factor of m:

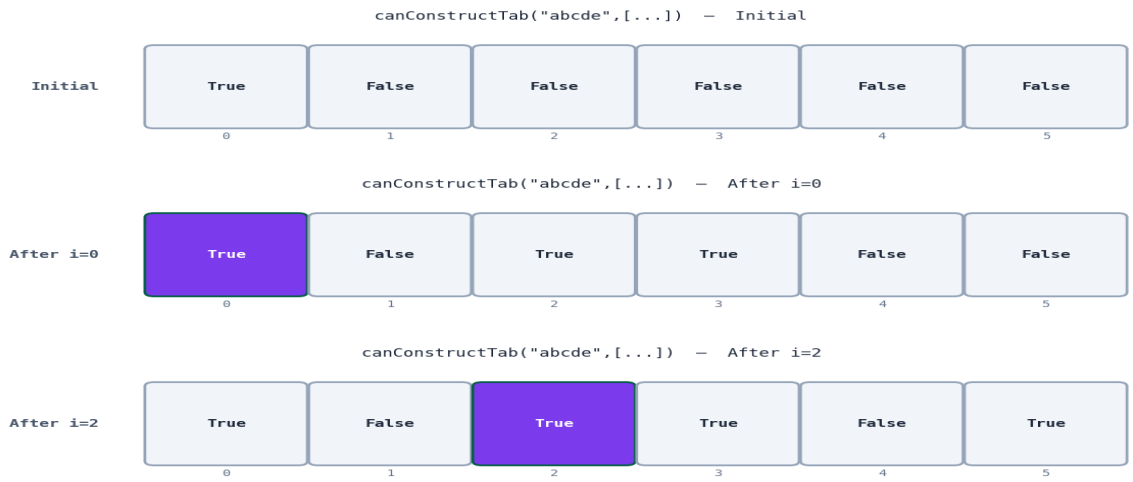
```
Brute-force: Time O(n^m · m) Space O(m²)

Memoization: Time O(n · m²) Space O(m²)

Tabulation: Time O(n · m²) Space O(m) ← table stores booleans only
```

Memoization Implementation

```
def can_construct(target, word_bank, memo=None):

if memo is None: memo = {}

if target in memo: return memo[target]

if target == "": return True # base case: empty string always constructible

for word in word_bank:

if target.startswith(word): # prefix check — critical rule

suffix = target[len(word):] # slice off the matched prefix

if can_construct(suffix, word_bank, memo):

memo[target] = True

return True

memo[target] = False

return False
```

Tabulation Walk-Through

The table is indexed by **position** in the string (0 = start, m = end). If `table[i]` is True and a word from the bank exactly matches `target[i : i+len(word)]`, we set `table[i + len(word)] = True`.




```
def can_construct_tab(target, word_bank):
m = len(target)
table = [False] * (m + 1)
table[0] = True

for i in range(m + 1):
if table[i]:
for word in word_bank:
end = i + len(word)
if end <= m and target[i:end] == word:
table[end] = True

return table[m]
```

■ Common Gotchas

- Prefix-only matching. Using "word in target" matches words in the middle — wrong.
- Memo key is the suffix string, not an index. This is fine but slightly less efficient than tabulation.
- Table index = position in string (not a value). Conceptual shift from numeric DP.
- String slicing is O(k) where k = slice length — this is why complexity gains an extra m factor.

Problem 5 — countConstruct (String Count Problem)

Problem Statement: Return the **number of ways** `target` can be constructed from `wordBank`.

Return type: `int`

What Changed from canConstruct?

Instead of returning `True` on the first valid path, we must **count every valid path**. The base case `target=""` now returns `1` (one way: the empty combination). Dead ends return `0`. A running total `total_count` accumulates the contributions from all branches — **no early return**.

Recursive Tree

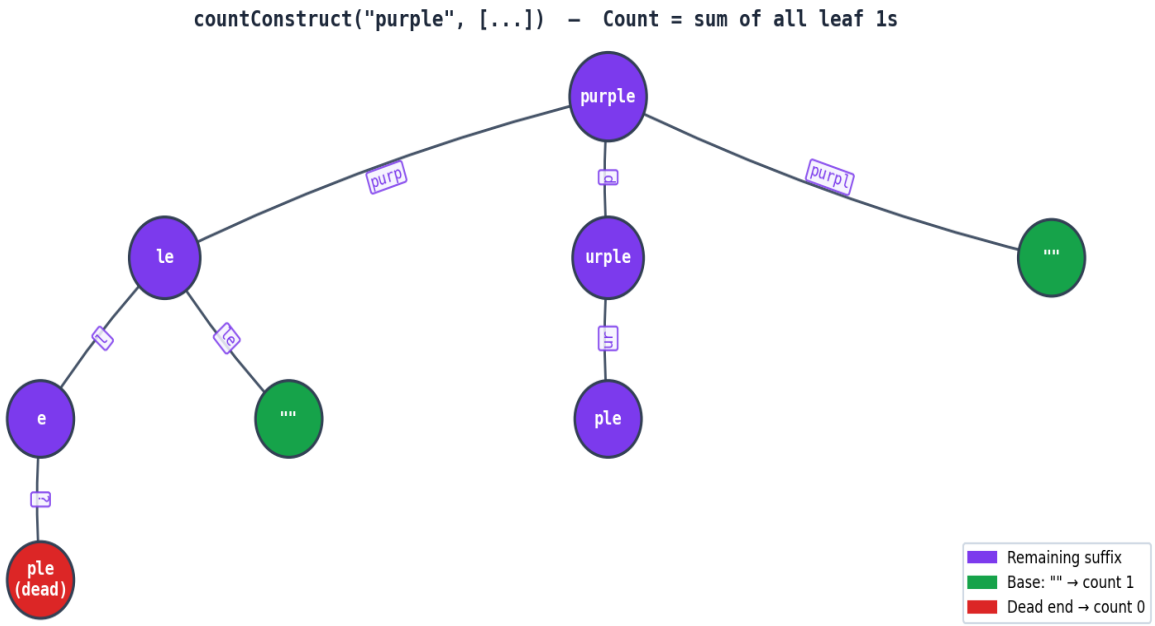


Figure 5. countConstruct("purple", ["purp","p","ur","le","purpl"]). Two branches each yield a leaf of 1; their sum gives the final count of 2.

Complexity Analysis

```
Brute-force: Time O(n^m · m) Space O(m²)

Memoization: Time O(n · m²) Space O(m²)

Tabulation: Time O(n · m²) Space O(m) ← stores ints, not lists
```

Memoization Implementation

```
def count_construct(target, word_bank, memo=None):

if memo is None: memo = {}

if target in memo: return memo[target]

if target == "": return 1 # one way to make the empty string

total_count = 0 # accumulate — no early return!
```

```
for word in word_bank:
if target.startswith(word):
suffix = target[len(word):]
count_for_rest = count_construct(suffix, word_bank, memo)
total_count += count_for_rest # add child's count

memo[target] = total_count
return total_count
```

The Tabulation Multiplier Effect

Instead of assigning `True`, we **increment** the future position by the count stored at the current position. If `table[i] = 3`, it means there are 3 distinct ways to reach position *i*. Every position reachable from *i* inherits all 3 of those ways:

```
def count_construct_tab(target, word_bank):
m = len(target)
table = [0] * (m + 1)
table[0] = 1 # one way to construct the empty string

for i in range(m + 1):
if table[i] > 0:
for word in word_bank:
end = i + len(word)
if end <= m and target[i:end] == word:
table[end] += table[i] # accumulate, not assign!

return table[m]
```

■ Common Gotchas

- table[end] += table[i] — not = table[i]. Multiple paths converge; summing is mandatory.
- Base case is 1 (one way), not True. Every other cell starts at 0.
- Propagation multiplies: if table[i]=3, all reachable cells gain +3 (not just +1).
- No early return — we must count every path.

Problem 6 — allConstruct (String Exhaustive Problem)

Problem Statement: Return **all** combinations of words from `wordBank` that construct `target`, as a 2-D array. Return `[[]]` if target is empty. Return `[]` if impossible.

Return type: `list[list[str]]`

The Two Base Cases — A Critical Distinction

These two values look similar but mean completely different things and cannot be swapped:

```
[[]] → one valid way exists: the empty combination. Used when target == "".

[] → zero valid ways exist. Used when no prefix matches (dead end).
```

Returning `[]` from the base case would cause every valid combination in the entire tree to be silently discarded — the most dangerous bug in this problem set.

Recursive Tree

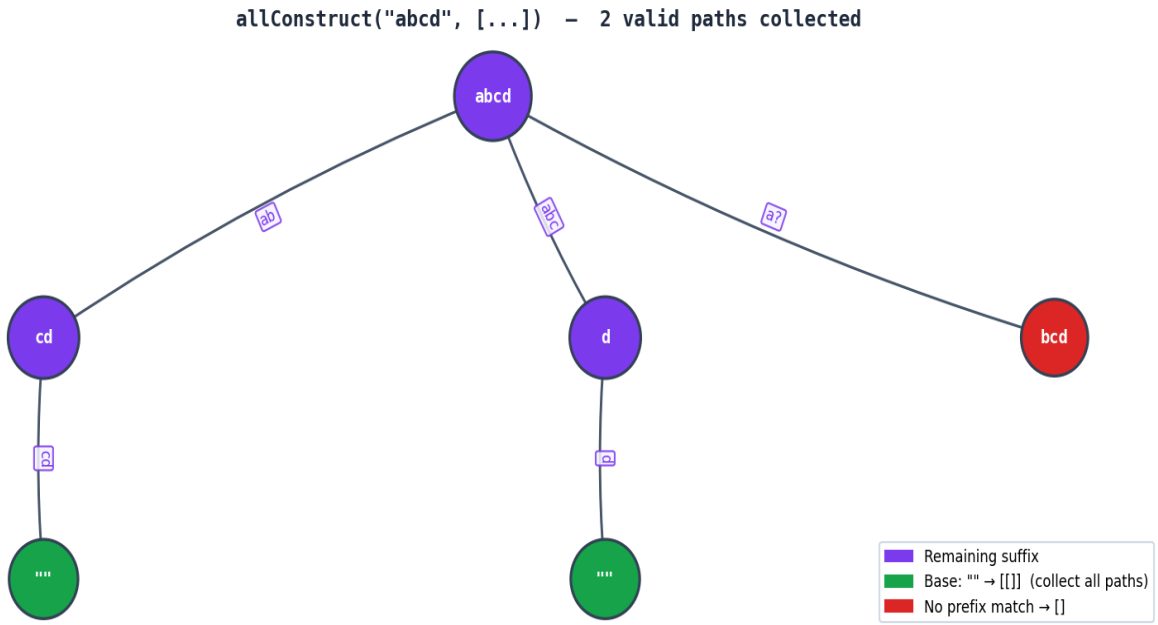


Figure 6. allConstruct("abcd", ["ab","cd","abc","d"]). Two paths reach the empty string. Each parent prepends its word to every sub-array returned by the child, assembling the final 2-D result.

Why Memoisation Cannot Help Asymptotically

For the previous five problems, memoisation reduced complexity from exponential to polynomial. Here, the **output itself can be exponentially large** — in the worst case every possible word sequence is valid. No algorithm can produce exponentially many combinations faster than exponential time. Memoisation avoids re-exploring identical suffixes, which helps in practice, but the asymptotic bound is unchanged:

```
Brute-force: Time O(n^m) Space O(n^m)

Memoization: Time O(n^m) Space O(n^m) — output dominates

Tabulation: Time O(n^m) Space O(n^m)
```

Memoization Implementation

```
def all_construct(target, word_bank, memo=None):
if memo is None: memo = {}
if target in memo: return memo[target]
if target == "": return [[]] # one way: the empty combination

all_ways = [] # collect EVERY valid path

for word in word_bank:
if target.startswith(word):
suffix = target[len(word):]
suffix_ways = all_construct(suffix, word_bank, memo)
# Prepend current word to each sub-array returned by child
target_ways = [[word, *way] for way in suffix_ways]
all_ways.extend(target_ways)

memo[target] = all_ways
return all_ways
```

Tabulation Implementation

The table is now a 3-D structure: a list of lists of lists. Each position *i* holds all combinations of words that bring us from the start of the string to position *i*.

```
def all_construct_tab(target, word_bank):
m = len(target)
table = [[] for _ in range(m + 1)] # unique empty lists — NOT [[]] * (m+1)!
table[0] = [[]] # one way to reach position 0: empty combo

for i in range(m + 1):
if table[i]:
for word in word_bank:
end = i + len(word)
if end <= m and target[i:end] == word:
new_combos = [combo + [word] for combo in table[i]]
table[end].extend(new_combos)

return table[m]
```

■ Common Gotchas

- [[]] vs [] — confusing these silently drops all valid combinations.
- [[] for _ in range(m+1)] NOT [[]] * (m+1). The latter aliases the same list m+1 times.
- Always copy: combo + [word]. Never mutate combo in place — future positions share references.
- No early return — by definition, we want every path.
- Memoisation gives no asymptotic benefit here. The output size is the bottleneck.

Pattern Taxonomy & Decision Guide

The six problems form a 2×3 grid across two axes: the **domain** (numeric subtraction vs. string prefix stripping) and the **return constraint** (decision / any path / optimal / count / all paths).

Return Constraint	Numeric DP	String DP
Boolean (any path?)	canSum	canConstruct
Any one path	howSum	—
Optimal (shortest) path	bestSum	—
Count of all paths	—	countConstruct
All paths (exhaustive)	—	allConstruct

The DP Recipe

1. Subproblem	What shrinks with each recursive call? (remainder int or suffix string)
2. Base cases	When do we stop? What do we return at each stopping condition?
3. Return type	bool / any-list / shortest-list / int count / list-of-lists
4. Early return?	Only if we need any one answer (canSum, howSum, canConstruct).
5. Memo key	The unique identifier of the subproblem (remainder or suffix).
6. Tabulation seed	What value goes at index 0? How does it propagate forward?

Memoization vs Tabulation

Memoization (Top-Down)	Tabulation (Bottom-Up)
Recursive	Iterative
Sparse subproblems: only computes what is needed	Computes all subproblems unconditionally
Risk of stack overflow on deep recursion (m large)	No recursion stack — safe for large m
Easier to derive: starts from brute-force	Requires inverting direction of thought
Memo key = subproblem identifier	Table index = subproblem identifier